

1교시 · 바이브코딩 입문 — 웹 앱을 만들고 LLM을 붙인다

배경 지식 (알아두면 좋은 맥락)

- **바이브코딩(Vibe Coding)**: 코드를 직접 타이핑하지 않고 자연어로 설명 → AI가 구현. 2025.02 Andrej Karpathy가 명명, Collins 사전 2025 올해의 단어.
- Google DORA 2025: AI 도입률 90%. Gartner: 2028년까지 기업 엔지니어 AI 코딩 도구 사용률 90% 전망.
- 개발자 역할 이동: 구현(implementation) → 오케스트레이션(문제정의·설계·품질검증).
- 문제점: DORA에 따르면 AI 도입은 처리량은 늘리지만 **배포 불안정성도 함께 높인다**. 도입률 90%인데 신뢰가 못 따라가는 게 현재 병목. → 성과는 도구가 아니라 조직의 워크플로/플랫폼 품질이 좌우.
- AI 코딩 도구 3단계: 웹 채팅창(복붙) → IDE 내 AI(제안) → **터미널 CLI(폴더 통째로 위임)**. Claude Code는 3번째 방식.

왜 Claude Code / 왜 터미널(PowerShell)인가

- 작업 범위가 **폴더 단위로 명확** (실행 위치가 곧 작업 공간, 바깥은 허락 없이 접근 안 함)
- 실행 전 **승인**을 요청함
- CLI는 AI가 실행 결과를 그대로 읽고 다음 판단에 씀 (글자로 주고받으니 자기 로그를 스스로 읽을 수 있음)
- 터미널에는 무엇을 읽고 무엇을 실행했는지 로그가 그대로 남음 (웹/앱은 화면이 가림)

설치 & 시작 (PowerShell)

```
irm https://claude.ai/install.ps1 | iex      # 설치 (관리자 권한 불필요)
claude --version                             # 확인
```

- 오늘 쓰는 PowerShell 명령은 **이동/확인**뿐: `cd` 폴더, `cd ..`, `ls`, `ls -Force` (숨김파일 포함, `.env` 확 인용)
- 실습 폴더로 이동 후 `claude` 실행 → Claude Code는 **실행된 위치를 작업 공간**으로 삼음 (영뚱한 위치에 서 실행하면 영뚱한 곳에 파일 생김)
- 최초 실행 시 브라우저 로그인 (배부 계정) — 한 번만 하면 됨

자주 쓰는 명령어

명령	용도
<code>/model</code>	모델 선택 (용도별로 제한하면 토큰 절약)
<code>/usage</code>	사용량 확인 (구독 한도 있음, 하루 한 번은 확인)

명령	용도
<code>/init</code>	프로젝트 훑어 <code>CLAUDE.md</code> 생성
<code>/clear</code>	대화 맥락 초기화 (만든 파일은 유지)
<code>/compact</code>	대화 맥락 요약 정리 (6교시에 사용)
<code>/resume</code>	이전 세션 복구
<code>/permissions</code>	권한 상태 확인
<code>Esc</code>	진행 중 작업 중단

API란 무엇인가 (실습으로 체감)

브라우저 주소창에 직접 입력해보기:

```
https://api.open-meteo.com/v1/forecast?latitude=37.5&longitude=127.0&current=temperature_2m
```

- `https://api.open-meteo.com/v1/forecast` = 창구 주소
- `?` 뒤 = 요청 조건 (`latitude` , `longitude` , `current` , `&` 로 연결)
- 응답은 사람용 문서가 아니라 **프로그램이 읽을 값 묶음**

오늘 쓰는 두 API/라이브러리:

- **Leaflet** — 지도 표시, 클릭한 위도·경도 반환 (공개 라이브러리)
- **Open-Meteo** — 위도·경도 → 날씨/예보 반환 (키 발급 불필요)

Claude Code가 프레임워크를 고르는 순서: ① 폴더 안 기존 스택 → ② `CLAUDE.md` 지시서 → ③ 요구사항 성격(정적페이지=HTML/CSS/JS, 상태관리 필요=프레임워크) → ④ 학습데이터 통용도(무난하지만 개선 없음). **스택을 고정하려면 지시문이나 CLAUDE.md에 명시해야 함** (오늘 Leaflet/Open-Meteo를 지정한 이유).

실습 1 — 지도 날씨 웹 앱

지도에서 위치를 클릭하면 그 지역의 현재 날씨와 앞으로 7일 예보를 보여주는 웹 앱을 만들어줘.
지도는 Leaflet을 쓰고, 날씨 데이터는 Open-Meteo API를 써줘.
다 만들면 브라우저에서 열어서 보여줘.

지시문 3요소: **무엇을 만들지**(결과 중심 서술) / **무엇으로 만들지**(라이브러리 지정, 안 하면 AI 임의 선택→팀마다 스택이 갈려 이후 실습이 안 맞음) / **끝나고 무엇을 할지**(브라우저 실행까지).

API Key와 보안 — .env

- API Key = 누가 호출했는지 식별하는 값. 비용은 Key 주인에게 청구됨.
- Key를 코드에서 분리해 .env 파일에 둬 → 코드/저장소 공유 시 Key 노출 방지 목적.

⚠ .env 가 막아주는 것 vs 못 막는 것

.env 는 Git 유출 방지용이지, Claude Code(Agent)로부터 Key를 숨기는 보안장치가 아니다. Agent가 프로젝트 파일/셸에 접근 가능하면 .env 나 환경변수 값을 읽을 수 있음.

- 개인 실습: .env + .gitignore + 프로젝트별 Key + 사용량 제한
 - 강화: OS Keychain / Credential Manager / Secret Manager
 - 운영/기업: Claude Code에는 Key를 주지 않고, 별도 API Proxy/Gateway가 Key를 보관·대행 호출
- 핵심: Agent로부터 Secret을 보호하려면 Agent가 Secret을 직접 가지지 않는 구조가 필요.

.env 만들기 (Claude Code에게 시킴):

이 폴더에 .env 파일을 만들고 OPENAI_API_KEY 항목을 넣어줘.
값은 비워 두고, 내가 직접 채울게.
그리고 .gitignore 에 .env 를 추가해줘.

→ 메모장으로 열어 값 직접 입력 (파일명이 .env.txt 로 저장되지 않게 확장자 표시 확인)

실습 2 — 채팅 기능 추가

.env 의 OPENAI_API_KEY 를 환경변수로 읽어서 쓰고,
지금 보고 있는 지역의 날씨 데이터를 바탕으로
"오늘 우산 필요해?" 같은 질문에 답해주는 채팅창을 웹 앱에 추가해 줘.
모델은 gpt-5.6-luna 를 써 줘.
키 값은 코드에 직접 쓰지 마.

이 키는 하루 종일 재사용 — 이후 앱들도 각자 폴더에서 .env.example 을 복사해 같은 키를 넣는다.

앱 개선 (시간 남으면)

"예보를 그래프로", "마커 여러 개로 지역 비교", "낮/밤 배경 전환", "채팅 답변 근거 표시" 등을 하나씩 지시하고 고치는 데 걸린 시간을 기록 — 오늘 강조점은 만드는 시간보다 고치는 시간.

정리

- 도구 구분: Claude Code(개발) vs OpenAI GPT(앱 내 AI) — 8교시까지 유지
- .env 는 Git 유출만 막고, Agent로부터 Secret을 숨기지 못한다
- 다음 질문: 이 AI는 우리 회사 문서 안의 내용도 알고 있을까? → 2교시 RAG로 이어짐